

计算机体系结构第一次实验报告

五级流水复习和分析

2312966 林晖鹏 计算机学院

一、实验目的

1. 修改原始代码source code中的bug，保证指令运行正确
2. 针对五级流水线中新增加的指令，逐级分析指令执行过程和执行结果，梳理流水线知识点

二、运行code

代码的bug同上一学期最后一次实验一样，主要是需要修改一下IP的配置（如下图）：

inst_rom的配置：

Component Name inst_rom

Basic

Port A Options

Other Options

Summary

Interface Type Native ☐ Generate address interface with 32 bits

Memory Type Single Port ROM ☐ Common Clock

ECC Options

ECC Type No ECC ☐ Error Injection Pins Single Bit Error Injection

Write Enable

☐ Byte Write Enable

Byte Size (bits) 9

Component Name inst_rom

Basic

Port A Options

Other Options

Summary

Memory Size

Port A Width 32 Range: 1 to 4608 (bits)

Port A Depth 256 Range: 2 to 1048576

The Width and Depth values are used for Read Operation in Port A

Operating Mode Write First Enable Port Type Always Enabled

Port A Optional Output Registers

☐ Primitives Output Register ☐ Core Output Register

☐ SoftECC Input Register ☐ REGCEA Pin

然后再上传一下coe文件：

Component Name inst_rom

Basic

Port A Options

Other Options

Summary

Pipeline Stages within Mux 0 Mux Size: 1x1

Memory Initialization

☒ Load Init File

Coe File sktop/tixijiegou/source_code/pipeline_inst/inst_pipeline.coe

☐ Fill Remaining Memory Locations

Remaining Memory Locations (Hex) 0

Remaining Memory Locations (Hex)

data_rom的配置：

Component Name

data_ram

Basic

Port A Options

Port B Options

Other Options

Summary

Interface Type

Native

☐ Generate address interface with 32 bits

Memory Type

True Dual Port RAM

☐ Common Clock

ECC Options

ECC Type

No ECC

☐ Error Injection Pins

Single Bit Error Injection

Write Enable

☒ Byte Write Enable

Byte Size (bits)

8

Component Name

data_ram

Basic

Port A Options

Port B Options

Other Options

Summary

Memory Size

Write Width

32

Range: 8 to 4096 (bits)

Read Width

32

Write Depth

256

Range: 2 to 1048576

Read Depth

256

Operating Mode

Write First

Enable Port Type

Always Enabled

Port A Optional Output Registers

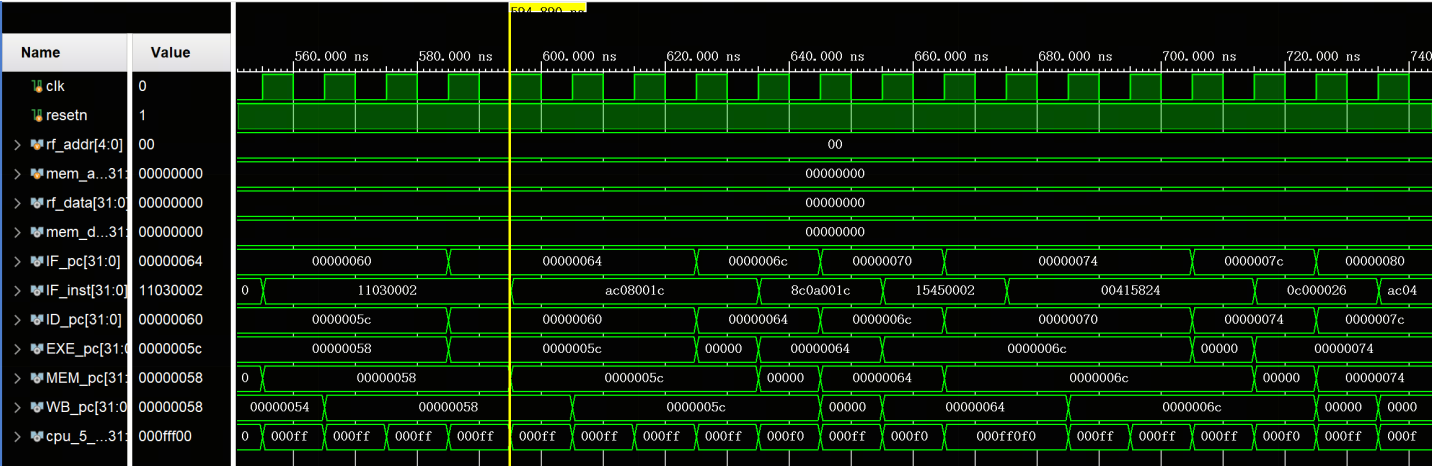
☐ Primitives Output Register

☐ Core Output Register

☐ SoftECC Input Register

☐ REGCEA Pin

然后导入testbench文件后运行Simulation，最终运行流水线成功！



三、新增指令分析

1. 流水线回顾

开始指令分析之前先回顾一下流水线的五个阶段，5 级流水线将指令执行拆分为**取指、译码、执行、访存、写回** 5 个阶段，每个阶段功能如下：

- **取指（IF）**：从指令存储器中，根据程序计数器（PC）的值，取出下一条要执行的指令。
- **译码（ID）**：解析指令的操作码（确定要执行的操作，如加法、乘法）和操作数（从通用寄存器中读取参与运算的值）。
- **执行（EXE）**：对操作数执行指令指定的运算（如算术运算、逻辑运算、地址计算等）。
- **访存（MEM）**：若指令是加载（load）或存储（store）指令，在此阶段访问数据存储器，读取或写入数据。
- **写回（WB）**：将执行阶段或访存阶段得到的结果，写回通用寄存器（如把运算结果存回寄存器，或把从内存加载的数据存回寄存器）。

接下来我们对表格中的新增指令进行分析：

表 9.3 5 级流水线 CPU 新增的指令

指令名称	汇编指令	功能描述
有符号乘法	mult rs,rt	(HI,LO)=sign(GPR[rs])*sign(GPR[rt])
从 LO 寄存器取值	mflo rd	GPR[rd]=[LO]
从 HI 寄存器取值	mfhi rd	GPR[rd]=[HI]
向 LO 寄存器存值	mtlo rs	[LO]=GPR[rs]
向 HI 寄存器存值	mt hi rs	[HI]=GPR[rs]
从 cp0 寄存器取值	mfc0 rt,cs	GPR[rt]=CPR[cs]
向 cp0 寄存器存值	mtc0 rt,cd	CPR[cd]=GPR[rt]
系统调用	syscall	CPR[14.0]=PC, CPR[13.0][6:2]=01000, CPR[12.0][1]=1, PC=CPR[15.1]+0x180
异常返回	eret	CPR[12.0][1]=0, PC=CPR[14.0]

注：CPR 表示协处理器 0 寄存器，CPR[12.0]表示 STATUS 寄存器，CPR[13.0]表示 CAUSE 寄存器，CPR[14.0]表示 EPC 寄存器。CPR[15.1]为 EBASE 寄存器，syscall 指令会使用该寄存器确认 Exception 的入口地址，但在本参考设计中未实现该寄存器，并设置 Exception 的入口地址为 0 地址。

2. mult rs, rt （有符号乘法）

mult rs, rt 是用于有符号整数乘法的指令，其结果存储在特殊寄存器对 HI （高位）和 LO （低位）中。

当pc为 000000ec 时，指令 mult \$12,\$29 进入流水线

2.1 IF取指

首先从coe文件中根据pc地址取出指令 019D0018 。

2.2 ID译码

此阶段译码指令并读取寄存器值：

- \$12 = 0x000C0000 = 786432 （十进制）
- \$29 = 0x000C000D = 786445 （十进制）

两者均为**正数**，所以是正常的有符号乘法。

预计乘积：

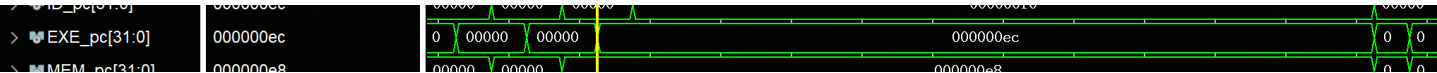
代码块

```
1 786432 × 786445 = 618,475,888,640 = 0x00000090_000F8000
```

所以预计HI最终为00000090，LO最终为000F8000。

2.3 EXE执行

在EXE执行阶段中，ALU 进行实际的有符号乘法运算。这里由于无法单周期内实现迭代乘法，所以这个阶段需要运行很多周期，因此会阻塞其他阶段的进行，下图时EXE阶段执行的PC，可以看到很长一段连续周期内，PC值都没有变化。



2.4 MEM访存

`mult` 指令**不访问数据存储器**。所以这里时空操作，不过仍然会进入这个阶段，只是不做任何操作。

2.5 WB写回

`mult` 指令**不直接写回通用寄存器**，而是将结果隐式写入特殊寄存器：两个特殊寄存器 HI 和 LO。

3. `mflo rd`（从 LO 寄存器取值）

`mflo rd` 用于将特殊寄存器 **LO** 的值传送到通用寄存器 `rd` 中。例如，当 `PC = 000000f0` 时，执行 `mflo $23` 将把该值写入 `$23`。

3.1 IF 取指

从指令存储器中根据当前 PC 地址取出指令机器码（对应 `mflo $23`）。

3.2 ID 译码

此阶段识别出指令类型为 `mflo`，确定目标寄存器为 `$23`，并设置控制信号以便后续将 LO 的内容写回寄存器堆。

3.3 EXE 执行

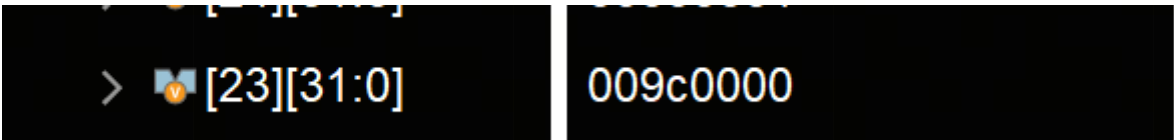
`mflo` 指令不进行算术运算，只在此阶段等待数据通路准备好。

3.4 3.4 MEM 访存

不访问数据存储器。

3.5 WB 写回

将特殊寄存器 LO 的值写入目标寄存器 `$23`，即 $\$23 \leftarrow LO$ ，`$23 = 009c0000`。



4. mfhi rd (从 HI 寄存器取值)

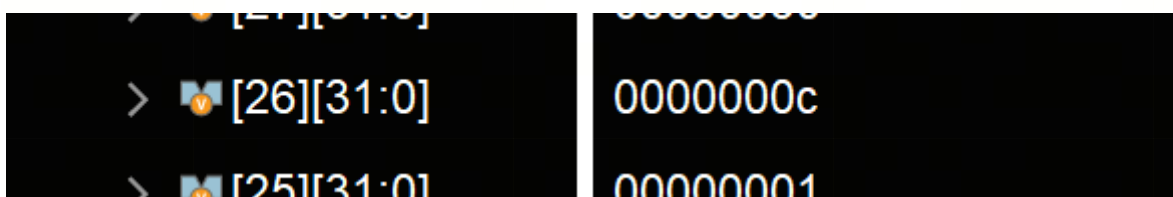
分析和上面的一模一样，这里不再赘述，放一个结果图片。



5. mtlo rs (向 LO 寄存器存值)

mtlo rs 用于将通用寄存器 rs 的值写入特殊寄存器 LO 中。

例如，当 PC = 000000f8 时，执行 mtlo \$26 后，LO = 0000000c。



5.1 IF 取指

从指令存储器取出 mtlo \$26 的机器码。

5.2 ID 译码

读取源寄存器 \$26 的值，并设置控制信号以将其写入 LO。

5.3 EXE 执行

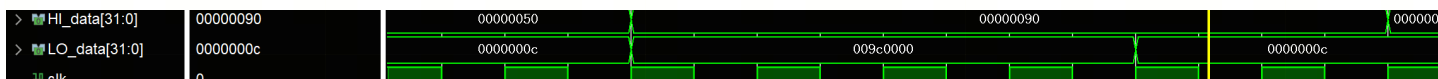
无算术计算，仅将 \$26 的值送往 HI/LO 通路，准备写入。

5.4 MEM 访存

不访问数据存储器。

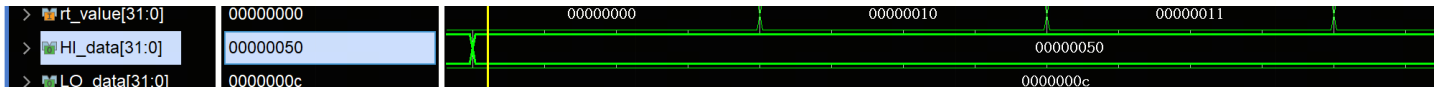
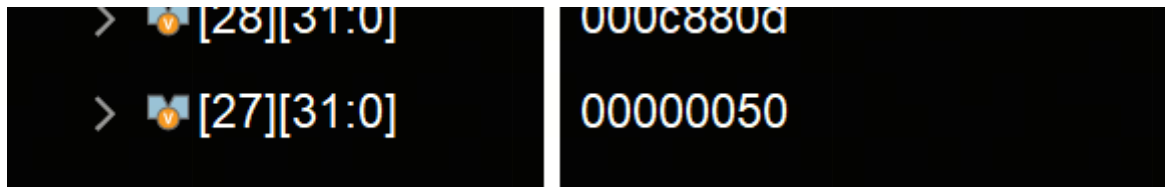
5.5 WB 写回

将 \$26 的值写入 LO 寄存器，即 $LO \leftarrow GPR[\$26] = 0000000c$ 。



6. mthi rs (向 HI 寄存器存值)

分析和上面的一模一样，这里不再赘述，放一个结果图片。



7. mfc0 rt, cs（从 CP0 寄存器取值）

`mfc0` 用于将协处理器 CP0 中寄存器 `cs` 的内容传送到通用寄存器 `rt`。

例如，当 `PC = 000000108` 时，执行 `mfc0 $2, c14` 会使 `GPR[¥2] ← CPR[c14]`。

7.1 IF 取指

根据 PC 取出 `mfc0` 指令。

7.2 ID 译码

识别为协处理器 0 读操作，确定目标寄存器 `$2` 与源寄存器索引 `c14`。

7.3 EXE 执行

无算术操作，等待 CP0 数据通路就绪。

7.4 MEM 访存

不访问主存。

7.5 WB 写回

将 CP0 中寄存器 `$14` 的值写回通用寄存器 `$2`。

8. mtc0 rt, cd（向 CP0 寄存器存值）

`mtc0` 用于将通用寄存器 `rt` 的值写入协处理器 CP0 寄存器 `cd`。

例如，`PC = 00000100` 执行 `mtc0 $0, c14` 会将 `$0` 的值写入 CP0 的 `$14` 寄存器（即 CAUSE 寄存器）。

8.1 IF 取指

取出 `mtc0 $0, c14` 的机器码。

8.2 ID 译码

读取 `$10` 的值并译码为 CP0 写操作。

8.3 EXE 执行

无算术运算，仅负责数据传递。

8.4 MEM 访存

不访问数据存储器。

8.5 WB 写回

将 `$0` 的值写入 CP0 寄存器 `c14`。

9. syscall（系统调用）

`syscall` 指令用于触发系统调用异常，它会将当前 PC 等信息存入协处理器 0 的相关寄存器，并跳转至异常入口地址。例如，`PC = 00000104` 的时候，执行 `syscall`。

9.1 IF 取指

取出 `syscall` 指令机器码。

9.2 ID 译码

译码阶段识别为系统调用异常指令，设置控制信号以产生异常。

9.3 EXE 执行

将当前 PC 写入 `CPR[14]`（EPC 寄存器），同时更新 `CPR[13][6:2] = 01000` 表示异常类型为系统调用。

9.4 MEM 访存

不访问数据存储器。

9.5 WB 写回

更新 `CPR[12.0][1] = 1`，跳转到异常入口地址 `PC = CPR[15] + 0x180`，开始异常处理。

104H	<u>syscall</u>	cp0[14.0] =0000_0104H, cp0[13.0][6..2] =01000B, cp0[12.0][1]=1, 跳转 Exception 入口地址，00H
------	----------------	--

由于难以查看cp0的状态，所以这里没有办法验证，但是下一条指令确实跳转到00H。



10. eret（异常返回）

eret 指令用于从异常处理程序返回，恢复之前保存的执行状态。

10.1 IF 取指

取出 eret 指令机器码。

10.2 ID 译码

译码识别为异常返回指令，准备恢复状态寄存器。

10.3 EXE 执行

将 CPR[12][1] 清零，表示退出异常模式。

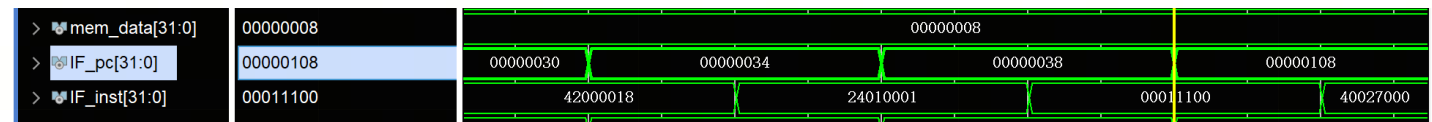
10.4 MEM 访存

不访问数据存储器。

10.5 WB 写回

将 PC 恢复为异常前的地址，即 $PC \leftarrow CPR[14]$ ，从 EPC 所指位置继续执行用户程序。

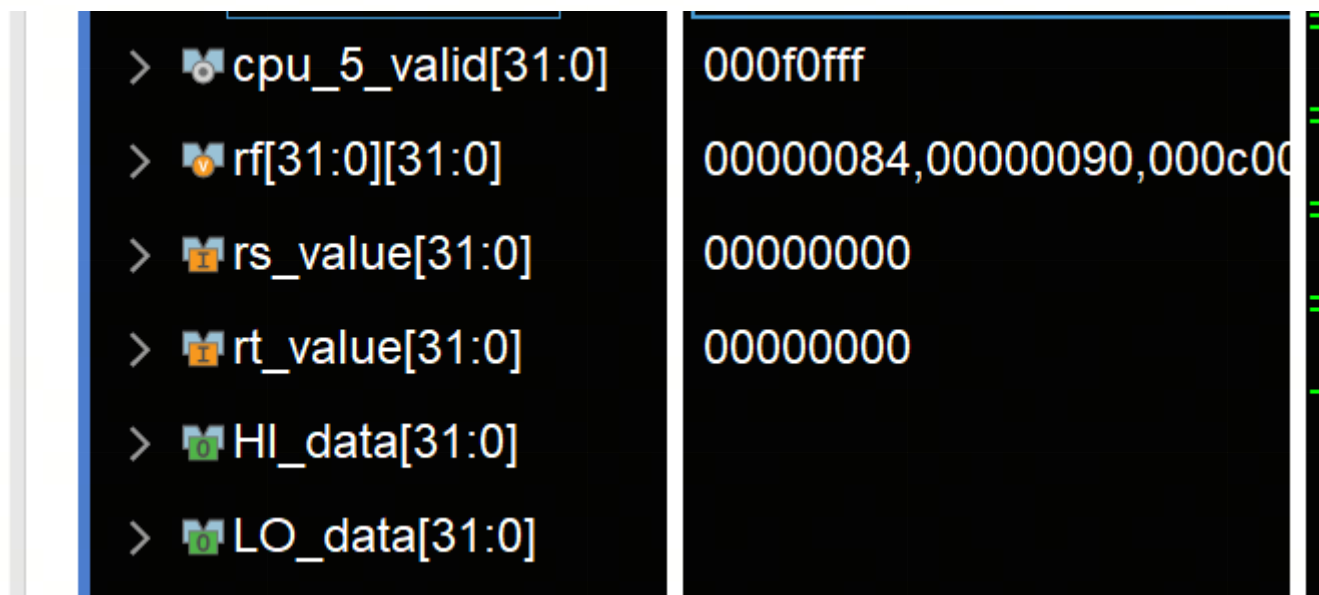
这里验证发现在30H执行完之后，确实跳转回之前存入的异常地址108H。



实验心得：

- 首先是感觉士别三日，已经完全不熟悉vivado的使用了，重新开始配置project文件的时候真的一头雾水，于是重新回去看以前的报告。卡在simulation很久，最后发现是因为tb没有放在top导致lcd的moudel一直找不到。

- 在实验过程中，想像以前一样查看一些具体数据的值，但是如果不在一开始就拖入仿真之中，就会产生空值无法查看，导致每次都要重新刷新一次，相当麻烦，不知道什么问题。



- 在实验过程中不知道如何查看cp0协处理器的寄存器，所以这里难以找到结果对照。
- 在本次实验的流程中，一条一条指令过，重新回顾了流水线的全过程，同时认识了新的指令。